

ROS 2 Humble + Gazebo Tutorial

From Beginner to Autonomous Robot Simulation

Author

Krishna Maithreya

1. Introduction

This tutorial provides a complete step-by-step guide for learning ROS 2 Humble and Gazebo simulation from the ground up. The objective is to progress from basic ROS communication to full robot simulation, navigation, SLAM, and eventually PX4 integration.

By the end of this tutorial, readers will be able to:

- Install ROS 2 Humble
 - Understand ROS architecture
 - Create ROS packages
 - Develop publishers and subscribers
 - Build robot models using URDF and Xacro
 - Simulate robots in Gazebo
 - Add sensors such as cameras, IMUs, and LiDARs
 - Perform SLAM and navigation
 - Integrate with PX4 and autonomous systems
-

2. System Requirements

Recommended Environment:

Component	Version
Ubuntu	22.04 LTS
ROS	Humble Hawksbill
Gazebo	Fortress
Python	3.10
Colcon	Latest

Verify Ubuntu:

```
lsb_release -a
```

Expected:

```
Ubuntu 22.04 LTS
```

3. Installing ROS 2 Humble

3.1 Add Universe Repository

```
sudo apt update
sudo apt install software-properties-common
sudo add-apt-repository universe
```

3.2 Add ROS Repository

```
sudo apt update
sudo apt install curl -y

sudo curl -sSL
https://raw.githubusercontent.com/ros/rosdistro/master/ros.key
-o /usr/share/keyrings/ros-archive-keyring.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/ros-archive-keyring.gpg]
http://packages.ros.org/ros2/ubuntu
$(. /etc/os-release && echo $UBUNTU_CODENAME) main" |
sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

3.3 Install ROS 2

```
sudo apt update
sudo apt install ros-humble-desktop -y
```

4. Environment Setup

Source ROS:

```
source /opt/ros/humble/setup.bash
```

Make it permanent:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

Verify:

```
ros2
```

5. Understanding ROS 2 Architecture

ROS 2 consists of:

- Nodes
- Topics
- Services
- Actions
- Parameters
- DDS Middleware

Communication Flow:

Publisher → Topic → Subscriber

Example:

```
Camera Node  
  |  
  v  
/image_raw  
  |  
  v  
Vision Node
```

6. ROS 2 Communication Example

Terminal 1

```
ros2 run demo_nodes_cpp talker
```

Terminal 2

```
ros2 run demo_nodes_py listener
```

Observe message exchange.

7. ROS 2 Command Line Tools

List Nodes:

```
ros2 node list
```

List Topics:

```
ros2 topic list
```

Echo Topic:

```
ros2 topic echo /chatter
```

Topic Information:

```
ros2 topic info /chatter
```

Service List:

```
ros2 service list
```

8. Creating a Workspace

Create Workspace:

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws
```

Build:

```
colcon build
```

Source:

```
source install/setup.bash
```

Permanent Setup:

```
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
```

9. Creating Your First Package

```
cd ~/ros2_ws/src  
  
ros2 pkg create  
--build-type ament_python  
my_robot_pkg
```

Package Structure:

```
my_robot_pkg  
├─ package.xml  
├─ setup.py  
└─ my_robot_pkg
```

10. Publisher Node

Create:

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Publisher(Node):

    def __init__(self):
        super().__init__('publisher')

        self.pub = self.create_publisher(
            String,
            'robot_status',
            10
        )

        self.timer = self.create_timer(
            1.0,
            self.publish_msg
        )

    def publish_msg(self):
        msg = String()
        msg.data = "Robot Alive"
        self.pub.publish(msg)

def main():
    rclpy.init()
    node = Publisher()
    rclpy.spin(node)

if __name__ == '__main__':
    main()

```

11. Subscriber Node

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class Subscriber(Node):

    def __init__(self):
        super().__init__('subscriber')

        self.sub = self.create_subscription(
            String,
            'robot_status',

```

```
        self.callback,  
        10  
    )  
  
    def callback(self,msg):  
        self.get_logger().info(msg.data)  
  
def main():  
    rclpy.init()  
    node = Subscriber()  
    rclpy.spin(node)  
  
if __name__ == '__main__':  
    main()
```

12. Introduction to Gazebo

Gazebo is a physics-based robotics simulator.

Capabilities:

- Rigid body dynamics
- Sensors
- Terrain
- Multi-robot simulation
- ROS integration

13. Installing Gazebo Fortress

```
sudo apt update  
  
sudo apt install  
ros-humble-ros-gz  
ros-humble-ros-gz-bridge  
-y
```

Verify:

```
gz sim
```

14. Gazebo Architecture

Core Components:

- World
- Models
- Sensors
- Physics Engine
- Plugins

Simulation Pipeline:

```
World
|
Robot
|
Sensors
|
ROS Bridge
|
ROS Topics
```

15. Launching Gazebo

Example:

```
gz sim shapes.sdf
```

Observe:

- Cube
- Sphere
- Cylinder

16. ROS–Gazebo Bridge

Bridge Clock Topic:

```
ros2 run ros_gz_bridge parameter_bridge
/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock]
```

Verify:

```
ros2 topic echo /clock
```

17. Robot Description Using URDF

Example:

```
<robot name="my_robot">

<link name="base_link">

<visual>
<geometry>
<box size="1 1 0.5"/>
</geometry>
</visual>

</link>

</robot>
```

Save as:

```
robot.urdf
```

18. Understanding URDF Components

A robot consists of:

- Links
- Joints
- Inertial Properties
- Visual Geometry
- Collision Geometry

Relationship:

```
Link -- Joint -- Link
```

19. Xacro

Why Xacro?

- Reusable parameters
- Modular robot descriptions
- Easier maintenance

Example:

```
<xacro:property name="length" value="1.0"/>
```

Use:

```
<box size="${length} 1 0.5"/>
```

Generate URDF:

```
xacro robot.xacro > robot.urdf
```

20. Spawning Robots in Gazebo

Start Gazebo:

```
gz sim empty.sdf
```

Spawn:

```
ros2 run ros_gz_sim create  
-file robot.urdf  
-name my_robot
```

21. Sensors

IMU

```
<sensor type="imu">
```

Topics:

```
/imu
```

Camera

```
<sensor type="camera">
```

Topics:

```
/image_raw
```

LiDAR

```
<sensor type="gpu_lidar">
```

Topics:

```
/scan
```

22. TF Transform Tree

TF manages coordinate frames.

Example:

```
map
├── odom
│   ├── base_link
│   │   └── camera_link
```

Visualize:

```
ros2 run tf2_tools view_frames
```

23. RViz Visualization

Launch:

```
rviz2
```

Useful Displays:

- TF
 - RobotModel
 - Image
 - LaserScan
 - PointCloud2
 - Path
-

24. Differential Drive Robot

Components:

- Base Link
- Left Wheel
- Right Wheel
- Wheel Joints

Controller:

```
gazebo_ros2_control
```

Commands:

```
/cmd_vel
```

25. ros2_control

Purpose:

- Hardware abstraction
- Joint control
- Simulation control

Supports:

- Position Control
 - Velocity Control
 - Effort Control
-

26. SLAM Toolbox

Install:

```
sudo apt install ros-humble-slam-toolbox
```

Launch:

```
ros2 launch slam_toolbox online_async_launch.py
```

Output:

```
/map
```

Save Map:

```
ros2 run nav2_map_server map_saver_cli
```

27. Navigation2

Install:

```
sudo apt install ros-humble-navigation2  
sudo apt install ros-humble-nav2-bringup
```

Launch:

```
ros2 launch nav2_bringup navigation_launch.py
```

Modules:

- Localization
- Global Planner

- Local Planner
 - Controller
 - Recovery
-

28. Robot Simulation Workflow

```
URDF/Xacro
  ↓
Gazebo
  ↓
Sensors
  ↓
ROS Topics
  ↓
SLAM
  ↓
Navigation
  ↓
Autonomous Robot
```

29. Project Structure

Recommended Layout:

```
my_robot
├── description
├── worlds
├── launch
├── config
├── rviz
├── meshes
├── scripts
└── maps
```

30. Advanced Topics

After mastering the fundamentals:

Simulation

- Multi-Robot Gazebo

- Custom Worlds
- Plugins
- Physics Tuning

Perception

- OpenCV
- RealSense
- Visual Odometry
- AprilTags

Navigation

- Nav2
- Behavior Trees
- Dynamic Obstacles

Autonomous Systems

- PX4 SITL
- MAVROS
- Micro XRCE DDS
- VIO
- UAV Navigation

31. Learning Roadmap

Stage 1: ROS Basics

- Nodes
- Topics
- Services
- Actions

Stage 2: Robot Modeling

- URDF
- Xacro
- TF

Stage 3: Simulation

- Gazebo
- Sensors
- ros2_control

Stage 4: Perception

- Cameras
- LiDAR

- SLAM

Stage 5: Navigation

- Nav2
- Localization
- Planning

Stage 6: Autonomous Robotics

- PX4
- VIO
- Multi-Agent Systems
- Real-World Deployment

Conclusion

ROS 2 Humble and Gazebo provide a complete ecosystem for modern robotics development. By progressing through communication, robot modeling, simulation, perception, and navigation, a learner can build the foundation required for advanced autonomous systems including UAVs, UGVs, multi-robot systems, and PX4-based platforms.